



Routing Guards

Persistent Interactive | Persistent University



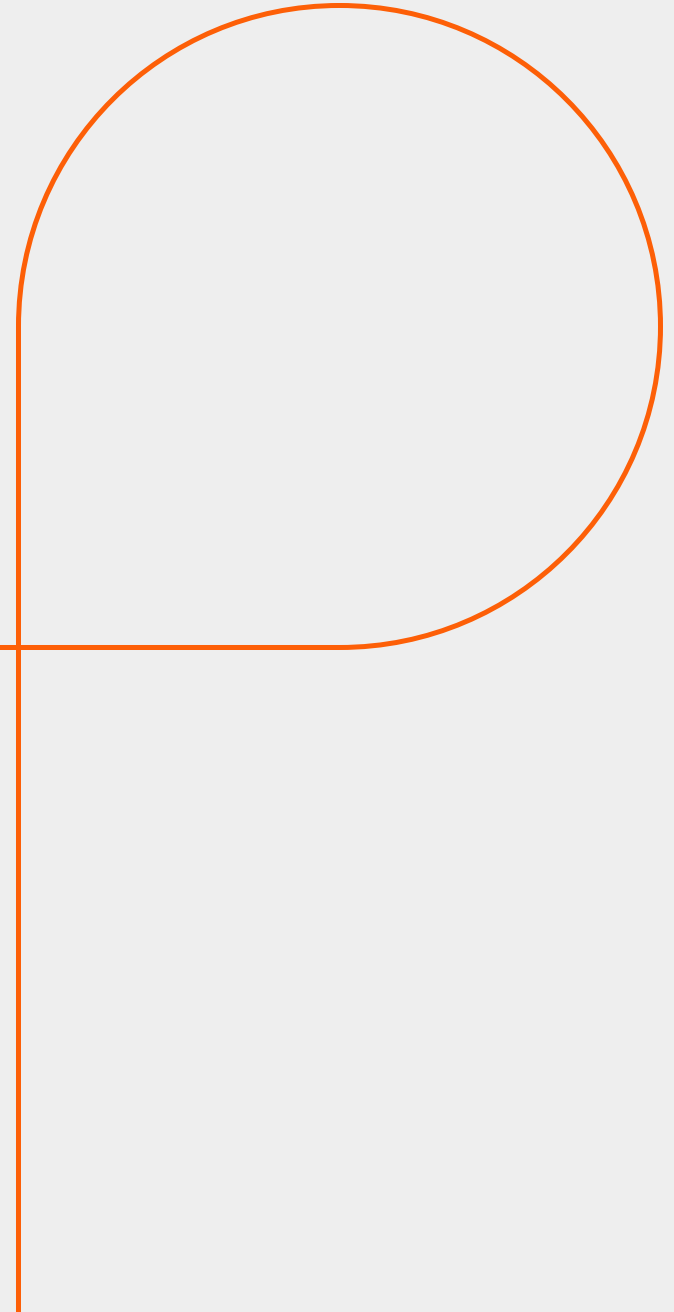
Key Learning Points

- Protecting routes using Guards
- Types of Guards
- Implementing Guards
- How to apply guards to the routes

Guard Types

- **CanActivate**
 - Checks to see if a user can visit a route.
- **CanLoad**
 - Checks to see if a user can route to a module that is lazy loaded.
- **CanActivateChild**
 - Checks to see if a user can visit a routes children.
- **CanDeactivate**
 - Checks to see if a user can exit a route.
- **Resolve**
 - Performs route data retrieval before route activation.

canActivate guard



canActivate guard

- A common use case in routing is, to check if user is authenticated before navigating to certain URLs.
- If not, then the user needs to be redirected to the Login page with appropriate message.
- Essentially, for certain routes we need to call some function which will check for the conditions and return true/false.
- If return value is true, user can navigate, else not.

Implementation

- This Guard is a class that implements **CanActivate** interface with logic written in method **canActivate()**.

```
@Injectable()
export class LoginGaurd implements CanActivate {

  canActivate(
    next: ActivatedRouteSnapshot,
    state: RouterStateSnapshot): boolean {
    return this.checkLogin();
  }
}
```

- Can generate the guard file using Angular CLI or with VS code plugins.

Guard function parameters

- To help in determining whether or not a guard should accept or deny access the guard function can be passed certain arguments:
 1. **component:** Component this is the component itself.
 2. **route:** `ActivatedRouteSnapshot` — this is the future route that will be activated if the guard passes, we can use its `params` property to extract the route params.
 3. **state:** `RouterStateSnapshot` — this is the future `RouterState` if the guard passes, we can find the URL we are trying to navigate to from the `url` property.

Provide the guards

- Add the guards to the list of providers in AppModule.

```
@NgModule({  
  declarations:[],  
  Imports:[],  
  providers: [LoginGuard]  
})  
export class AppModule { }
```

Applying the guard

- Apply the guard in the routing configuration.

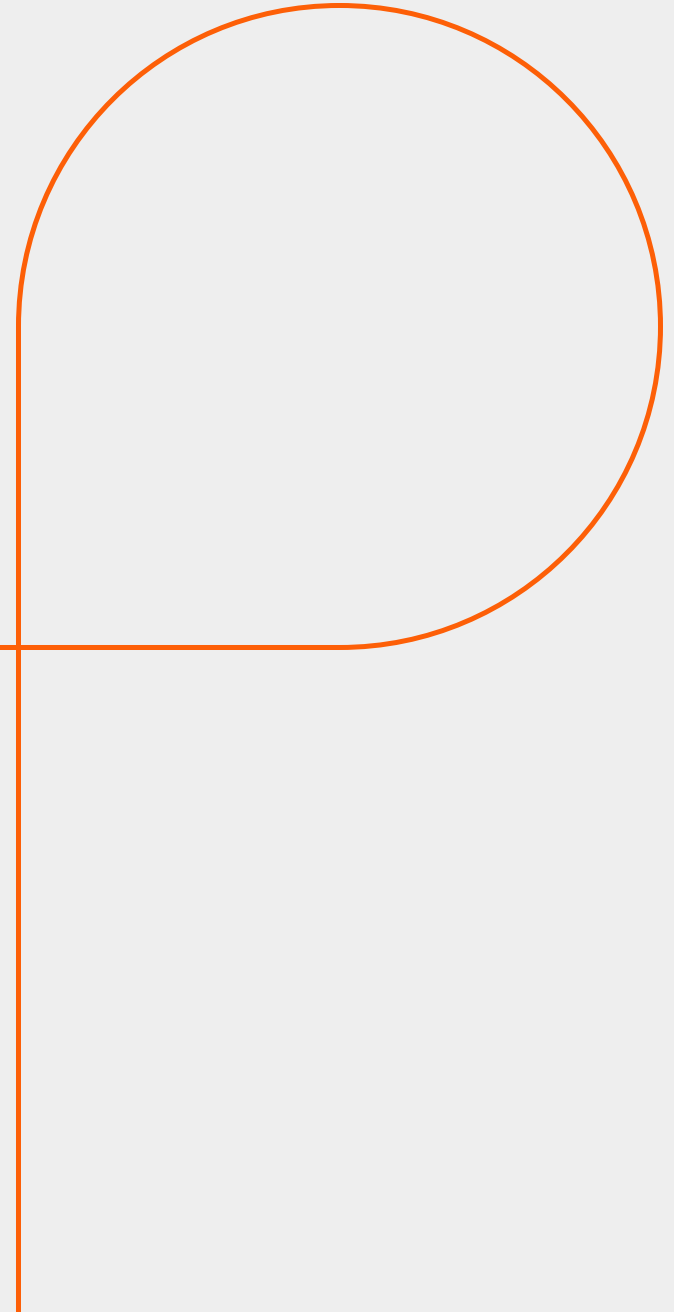
```
let routes:Routes=[  
  {  
    path:'users',  
  
    component:AllUsersComponent,  
    canActivate:[LoginGuard]  
  }  
]
```

- If LoginGuard returns true, then user is navigated to AllUsersComponent else not.

Redirection from a guard

- If we want to redirect users to a login page we may inject Router into the constructor and then use the navigate function to redirect them to the appropriate login page.

canLoad guard



Lazy loaded module with guard

- When CanActivate guard is applied on a lazy loaded module, then even when the guard returns false, the module gets loaded.

```
{  
  path:'products',  
  loadChildren:'./../productsModule/products.module#ProductsModule',  
  canActivate:[LoginGuard]  
}
```

- Here, even when LoginGuard returns false i.e user is not logged in, the ProductsModule gets loaded which is incorrect.
- To prevent this, implement the CanLoad guard.

Implementation

- This Guard is a class that implements **CanLoad** interface with logic written in method **canLoad()**.

```
@Injectable()
export class LoadGuard implements CanLoad {

}

canLoad(
  next: ActivatedRouteSnapshot,
  state: RouterStateSnapshot): boolean {
    return this.checkLogin();
}
```

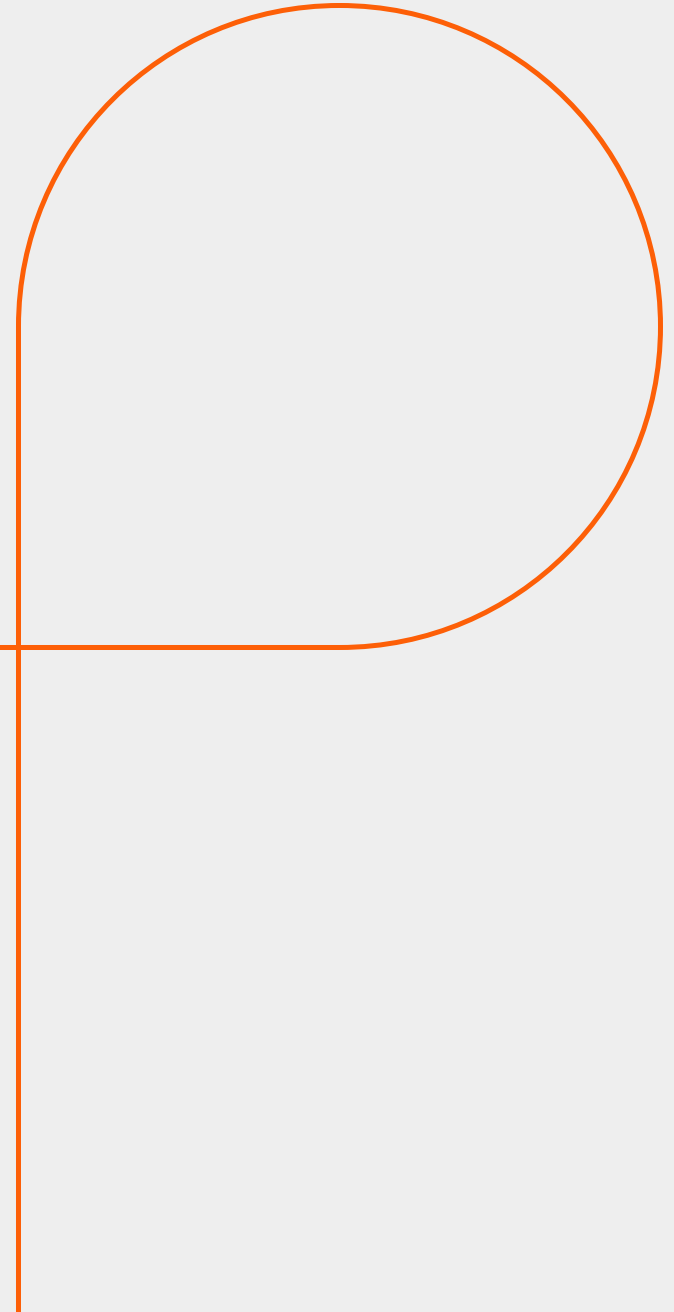
Applying the guard

- Apply the guard in the routing configuration.
- Don't load a the products module unless user is logged in.

```
{  
  path:'products',  
  loadChildren:'./../productsModule/products.module#ProductsModule',  
  canLoad:[LoadGuard]  
}
```

- If `LoadGuard` returns true, only then the `ProductsModule` is loaded else not.

canActivateChild guard



canActivateChild guard

- In certain cases, we might need to further check some conditions before user navigates to child routes of a primary route.
- Lets say the primary route is **/customerdashboard**
- Child routes are
 - **/customerdashboard/reports**
 - **/customerdashboard/history**
- There might be a need to check the role of the user before he can view the history. Eg: Only an admin user can view history while other users can view only reports.
- Another example is in banking sites, certain transactions request for profile password which is different from login password.
- In such cases, the **canActivateChild** guard comes in handy.

canActivateChild guard

- This guard works to check for conditions before navigating to child routes.

```
{  
  path: 'artist/:artistId',  
  component: ArtistComponent,  
  canActivateChild: [AlwaysAuthChildrenGuard],  
  children: [  
    {path: '', redirectTo: 'tracks'},  
    {path: 'tracks', component: ArtistTrackListComponent},  
    {path: 'albums', component:  
ArtistAlbumListComponent},  
  ]  
}
```

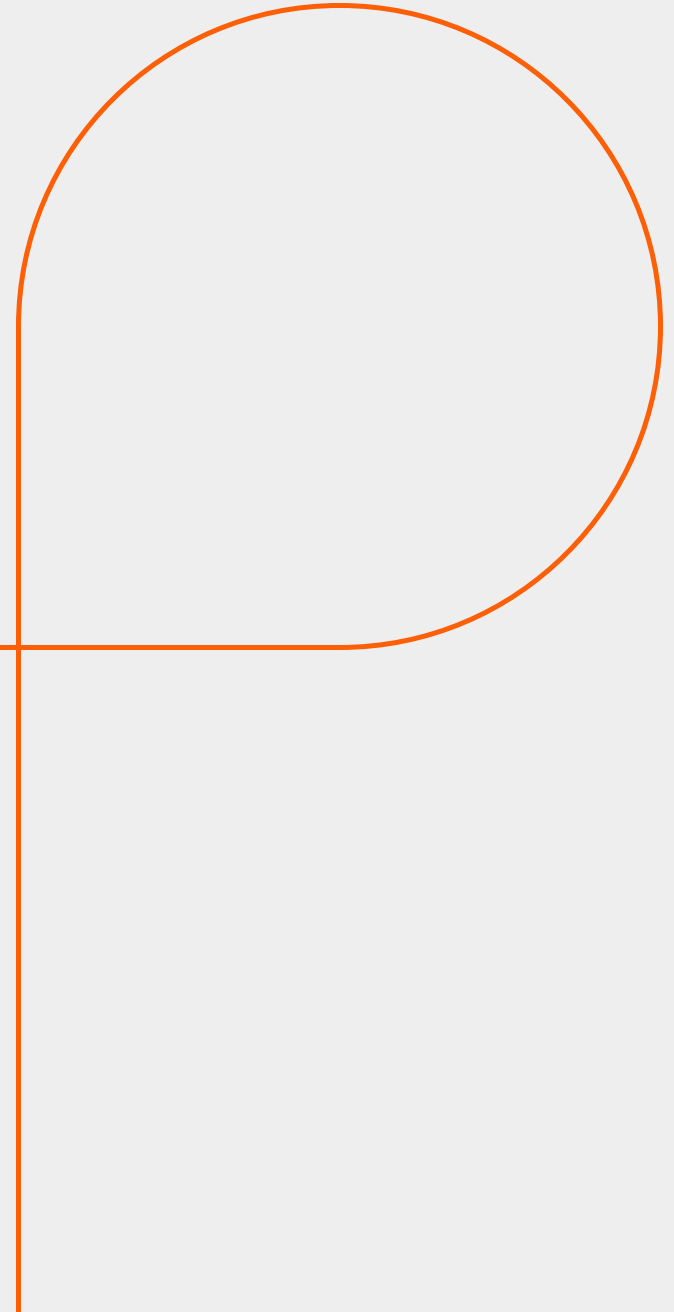
Implementation

- Implement the interface CanActivateChild

```
import {CanActivateChild} from "@angular/router";

class AlwaysAuthChildrenGuard implements
CanActivateChild {
  canActivateChild() {
    console.log("AlwaysAuthChildrenGuard");
    return true;
  }
}
```

canDeactivate guard



canDeactivate Guard

- This guard is used to warn people if they are navigating away from a page where they have some unsaved changes.
- Examples include un-submitted/unsaved forms, incomplete uploads.
- It can be used to take user confirmation whether he wants to really navigate away from the page or not.

Implementation

- The guard class has to implement the CanDeactivate interface and define the method canDeactivate().

```
import {CanDeactivate} from "@angular/router";  
  
class ConfirmDeactivateGuard implements CanDeactivate {  
  
}
```

Implementation

- Define the method `canDeactivate()`

```
canDeactivate(  
  component: SignUpFormComponent,  
  route: ActivatedRouteSnapshot,  
  state: RouterStateSnapshot  
): Observable<boolean> | Promise<boolean> | boolean {  
  return component.canDeactivate();  
}
```

The method in turn invokes the component's `canDeactivate()` method.

- `canDeactivate` method will return `Observable<boolean>` or `Promise<boolean>` or `boolean` .

canDeactivate() method in component

- The logic of checking if the page has unsaved changes or not lies in the component.

```
canDeactivate():boolean{  
  if(!this.isSaved)  
    return window.confirm('Are you sure you want to discard  
changes?');  
  else  
    return true;  
}
```

Applying the guard

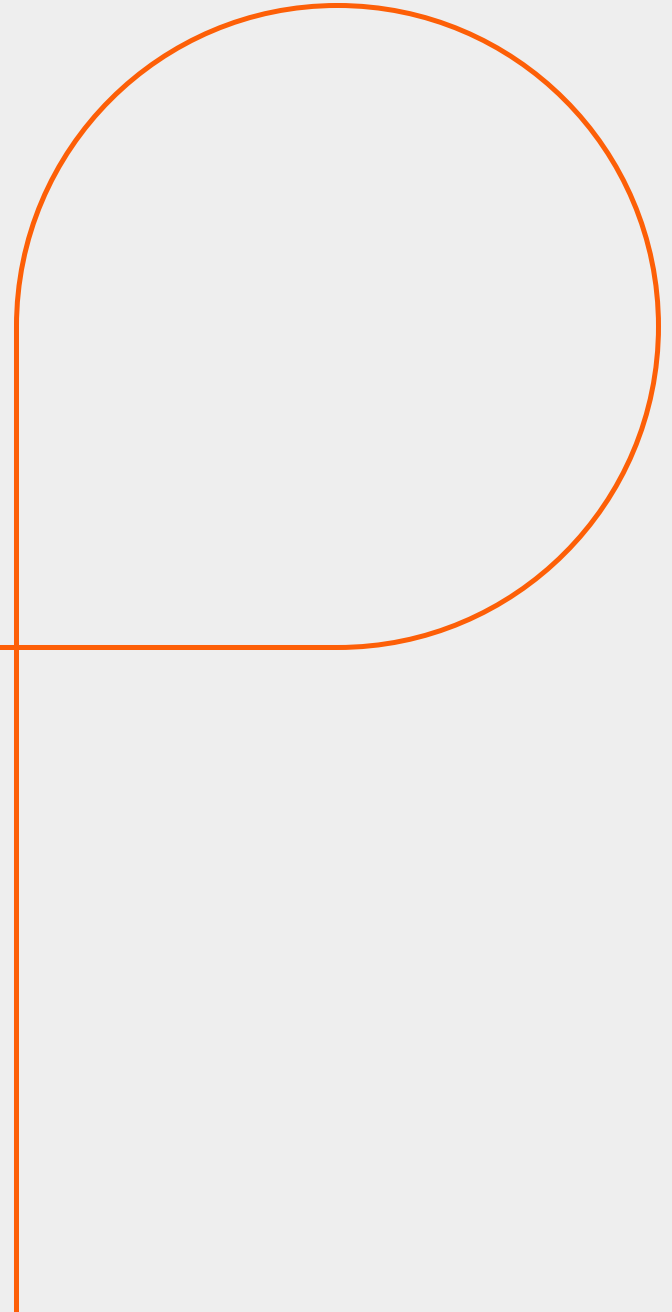
- Apply the guard in the routing configuration.

```
{  
  path:'signup',  
  component:SignUpFormComponent,  
  canDeactivate:[ConfirmDeactivateGuard]  
}
```

- Add the guard to the list of providers.

```
@NgModule({  
  providers: [ConfirmDeactivateGuard]  
})  
export class AppModule {}
```

Resolve guard



Need for Resolve guard

- Resolve guard is used in the scenario where before navigating to any route we want to ensure whether there is data available or not.
- Eg: Fetching user details in the UserDetailsComponent based on name.

```
export const AppRoutes: Routes = [  
  { path: 'user/:name', component:  
    UserDetailsComponent }  
];
```

Display the data

- Display of the user data in the UserDetailsComponent.

```
<h2>{{user.name}}</h2>  
<p>Phone:{{user.phone}}</p>  
<p>Address:{{user.address}}</p>
```

- There can be a delay in fetching this data especially through HTTP request.
- There is a possibility of getting an error when trying to display the data from an undefined or null object.

Safe Navigation operator

- One possible solution is to use safe navigation operator like below.

```
<h2>{{user?.name}}</h2>  
<p>Phone:{{user?.phone}}</p>  
<p>Address:{{user?.address}}</p>  
>
```

- The Safe Navigation Operator ensures that Angular won't throw any errors when we're trying to read from an object that is null or undefined.
- Although the error was avoided, the end user might observe some flickering of UI in such cases.
- This means we have to resolve data before navigating to that route.

Apply Resolver in the routing configuration

- The below ensures user is navigated to the UserDetailsComponent only when data is available.
- First the UserResolver is executed which returns a valid user object and then navigation occurs.

```
export const AppRoutes: Routes = [  
  {  
    path: 'user/:name',  
    component: UserDetailsComponent,  
    resolve: {  
      user: UserResolver  
    }  
  }  
];
```

Resolve guard Implementation

- The resolve guard implements the Resolve interface and defines the method resolve().
- The resolve method can return Observable or Promise or a synchronous value.

```
@Injectable()
export class ResolverGuard implements Resolve<T> {
    resolve(route: ActivatedRouteSnapshot, state: RouterStateSnapshot): Observable | Promise | <T>{

    }
}
```

- Resolver needs to be added to **providers** list in the module file.

Example

- Example of fetching User data from user service in the resolve guard.
- The username is being passed as a route param.

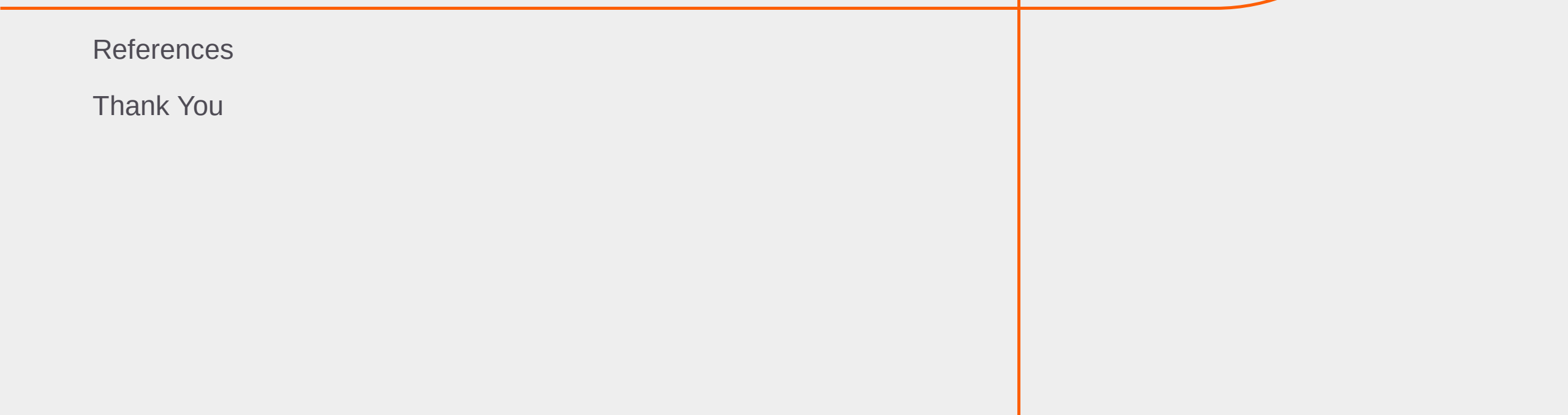
```
@Injectable()
export class UserResolverGuard implements Resolve<User> {
    resolve(route: ActivatedRouteSnapshot, state: RouterStateSnapshot):
    User{
        let name = route.paramMap.get('name');
        return this.userservice.getUser(name);
    }
}
```

Summary: Session

With this we have come to an end of our session, where we discussed about

- Types of Routing Guards
- Interfaces to implement Guards
- How to apply guards in the routing configuration

Appendix



References

Thank You

Reference Links

- <https://blog.thoughttram.io/angular/2016/10/10/resolving-route-data-in-angular-2.html>
- <https://www.concretepage.com/angular-2/angular-resolve-guard-example>
- <https://www.concretepage.com/angular-2/angular-candeactivate-guard-example>
- <https://angular.io/guide/router#milestone-6-asynchronous-routing>

Key Contacts :

Persistent University :

- Archana Ghatigar
archana_ghatigar@persistent.co.in



Thank you!

Persistent Interactive | Persistent University

